PROJET QUEST-AR : SURVIE & ÉNIGMES AR

RAPPORT COMPLET D'ARCHITECTURE & RÉPARTITION TECHNIQUE DES TÂCHES

Ce dossier technique constitue le document officiel de validation du projet **QuestAR**. Il a pour objectif de présenter de manière rigoureuse, transparente et vérifiable la contribution individuelle de chaque membre de l'équipe, les fichiers sources développés, ainsi que l'architecture technologique mise en place sous Unity avec l'AR Foundation.

Membre de l'Équipe	Rôle & Responsabilité Principale	Module & Périmètre Technique	Statut
Negzaoui (Lead Auth)	Authentification Biométrique VisageID & Gestion de Session Joueur	Capture Webcam/Selfie, MongoDB Atlas, Session statique, HUD Joueur	**100% Validé**
Amin	Système d'Armes FPS, Munitions & Balistique de Dégâts	Contrôles tactiles (Tir/Visée), Swap d'armes, Chargeurs, 35 dégâts/balle	**100% Validé**
Souhaila	Scène d'Énigme, Reliques Holographiques & Clavier Numérique	7 Reliques spatiales (360°), Détection Regard (Gaze), Clavier interactif	**100% Validé**
Mouhamed	Lampe Flash, Capteur de Luminosité & Gel des Zombies (Stun)	Flash Stun 5s, Cooldown 10s, Capteur Lux physique + Fallback ARCore	**100% Validé**
Nessim	Capteur Microphone, Détection du Sol AR & Course des Zombies	Détection sonore RMS (dB), Sprint des zombies (4.2 m/s), Rejet des tables	**100% Validé**

1. ARCHITECTURE GLOBALE DU JEU & PROGRESSION PAR NIVEAU

Le jeu **QuestAR** combine deux boucles de gameplay complémentaires : une phase de combat FPS en Réalité Augmentée dans la pièce du joueur (ARGameplay.unity), suivie d'une phase de réflexion cryptique holographique (Mission5_Enigme.unity) qui permet de débloquer l'équipement suivant. L'ensemble du flux est piloté de façon centralisée par le gestionnaire **QuestARLevelManager.cs**.

Niveau	Phase de Combat (ARGameplay)	Phase d'Énigme (Mission5_Enigme)	Récompense Débloquée
Niveau 1	• Pistolet seul (10 balles/chargeur) • 3 Zombies à éliminer • Marche lente (0.85 m/s)	• 3 Reliques générées (2 vraies + 1 leurre) • Code à **2 chiffres : [4 - 7]**	**Fusil d'Assaut** (Deuxième arme + bouton SWAP activé)
Niveau 2	• Pistolet + Fusil d'Assaut • 5 Zombies à éliminer • Zombies plus agressifs	• 5 Reliques générées (3 vraies + 2 leurres) • Code à **3 chiffres : [4 - 7 - 2]**	**Lampe Flash + Capteur Lumière** (Stun 5s + Cooldown 10s)

Niveau 3	• Pistolet + Fusil + Flash Stun • 7 Zombies à éliminer	• 7 Reliques générées (4 vraies + 3 leurres) • Code à **4 chiffres : [4 - 7 - 2 - 9]**	**Capteur Microphone** (Écoute des bruits ambiants et cris)
Niveau 4 (Final)	• Arsenal complet (Pistolet, Fusil, Flash, Micro) • 10 Zombies en horde • Si le joueur crie au micro -> Les zombies **courent** (4.2 m/s) !	*Vague ultime de survie*	**VICTOIRE TOTALE !** Écran royal de fin de jeu avec tableau de bord des statistiques

2. FICHES TECHNIQUES DÉTAILLÉES PAR MEMBRE DE L'ÉQUIPE

2.1 NEGZAOUI — AUTHENTIFICATION VISAGEID & GESTION DE SESSION

Rôle : Conception du système de connexion sécurisé, reconnaissance faciale par caméra, fallback de mot de passe, liaison MongoDB Atlas et maintien de l'identité du joueur à travers toutes les scènes.
Emplacement des fichiers :
• Scène : Assets/VisageID_Auth/Scenes/VisageID_Auth_Scene.unity
• Contrôleur UI : Assets/VisageID_Auth/Scripts/UI/VisageAuthUIController.cs
• Base de données : Assets/VisageID_Auth/Scripts/Database/MongoAtlasService.cs
• Modèles & Session : Assets/VisageID_Auth/Scripts/Models/VisageAuthModels.cs
• Affichage HUD Combat : Assets/_Project/Scripts/UI/ARPlayerSessionHud.cs

Fonctionnement technique :
1. **Capture faciale & Vérification :** À l'ouverture de l'application, VisageAuthUIController active la webcam pour capter le visage du joueur. Les descripteurs sont envoyés via API REST à la base MongoDB Atlas (MongoAtlasService.cs). Un mode de secours (Nom d'utilisateur + Mot de passe) est prévu si l'éclairage ne permet pas la reconnaissance.
2. **Persistance de Session :** Une fois validé, VisageAuthSession.CurrentUser conserve en mémoire les métadonnées de l'utilisateur (nom, email, identifiant). Le nom du joueur est injecté dans le bandeau de combat supérieur : *'JOUEUR : [Nom]'*.
3. **Écran de Victoire & Déconnexion :** Lors de la victoire totale au Niveau 4, le nom du joueur est fièrement affiché sur le badge d'honneur. En cliquant sur 'DÉCONNEXION', VisageAuthSession.ClearSession() purge les données de session, réinitialise l'orientation de l'écran en portrait et recharge la scène d'authentification.

2.2 AMIN — SYSTÈME D'ARMES, MUNITIONS, RECHARGEMENT & DÉGÂTS FPS

Rôle : Gestion de l'arsenal FPS adapté au tactile mobile, bascule pistolet/fusil, tirs raycast, chargeurs et balistique.
Emplacement des fichiers :
• Tir & Contrôle mobile : Assets/_Project/Scripts/Combat/ARInfimaPlayerShooter.cs
• Changement d'arme : Assets/_Project/Scripts/Combat/ARInfimaWeaponSwitcher.cs
• Raccordement Caméra AR : Assets/_Project/Scripts/Combat/ARInfimaWeaponRig.cs
• Configuration Arme : Assets/Infima Games/Low Poly Shooter Pack - Free Sample/Code/Weapons/Weapon.cs
• Dégâts Zombie : Assets/Scenes/HamaZombieAI.cs (Méthode TryProcessProjectileHit)

Fonctionnement technique :
1. **Gestion des Armes & Déblocage :** Au Niveau 1, le joueur commence avec le pistolet (chargeur de 10 balles). Le bouton tactile **SWAP** reste masqué. Dès que l'énigme du Niveau 1 est résolue, QuestARLevelManager.IsSecondWeaponUnlocked devient vrai : le bouton SWAP apparaît et permet d'alterner instantanément entre le Pistolet et le Fusil d'Assaut.
2. **Balistique & Hit Detection :** Chaque pression sur le bouton tactile **FIRE** déclenche Fire(). Une munition physique est instanciée avec un effet de bouche (Muzzle Flash) et un son percutant. Un cooldown de 0.06s empêche les tirs fantômes de rebondir entre plusieurs cibles.
3. **Calcul des Dégâts :** Chaque impact inflige exactement **35 points de dégâts** au zombie (TakeDamage(35f)). Les zombies possédant 100 points de vie, il faut 3 balles bien placées pour en éliminer un. La balle est détruite à l'instant de l'impact pour garantir 1 balle = 1 coup.

2.3 SOUHAILA — SCÈNE D'ÉNIGME HOLOGRAPHIQUE & SCEAU DU VAMPIRE

Rôle : Conception de l'univers de l'énigme, disposition des 7 reliques 3D en cercle (360°), lecture par visée/toucher, clavier tactile interactif, narration vocale et distribution des récompenses.

Emplacement des fichiers :

- Scène : Assets/QuestAR/Scenes/Mission5_Enigme.unity
- Panneau Holographique : Assets/QuestAR/Scripts/HoloPuzzlePanel.cs
- Reliques Flottantes : Assets/QuestAR/Scripts/IndicesFlottants.cs
- Clavier & Code : Assets/QuestAR/Scripts/CodePuzzle.cs
- Carte de Récompense : Assets/QuestAR/Scripts/CarteIndice.cs
- Narrateur Vocal : Assets/QuestAR/Scripts/EnigmeVoiceNarrator.cs

Fonctionnement technique :

1. **Génération des Reliques en AR :** Autour du joueur sont disposées des reliques holographiques surmontées d'un chiffre 3D géant. Chaque relique pivote pour faire face à la caméra du joueur (LookRotation).
2. **Logique Chronologique & Pièges :** L'histoire raconte la légende du seigneur vampire Aldric. Les vraies reliques se suivent dans le temps :
   - Rang 1 : Le Calice de Sang (Chiffre 4)
   - Rang 2 : La Dague d'Argent (Chiffre 7)
   - Rang 3 : Le Médaillon Noir (Chiffre 2)
   - Rang 4 : Le Grimoire Interdit (Chiffre 9)

Des indices 'Leurres' sont mélangés : ils tremblent quand on les fixe (ind.tremble) et contiennent des textes contradictoires.

3. **Clavier Numérique & Récompense :** Toucher le grand panneau central ouvre le clavier. Les cases se centrent dynamiquement selon la longueur du code (2, 3 ou 4 cases). La saisie du bon code déclenche l'ouverture spectaculaire du coffre holographique et active le bouton *'PASSER AU NIVEAU SUIVANT'*.

2.4 MOUHAMED — LAMPE FLASH, CAPTEUR DE LUMINOSITÉ & GEL DES ZOMBIES (STUN)

Rôle : Intégration du capteur de lumière physique et ARCore, modulation de la puissance lumineuse, gel des zombies pendant 5s, cooldown de sécurité de 10s.
Emplacement des fichiers :
• Gestionnaire Flash & Capteur : Assets/Scenes/HamaFlashStunManager.cs
• Bouton d'action tactile : Assets/_Project/Scripts/Combat/ARMobileCombatControls.cs

Fonctionnement technique :
1. **Capteur de Lumière Intelligent (Ne reste jamais à 0 Lux) :** Le script lit en priorité le capteur physique du smartphone via LightSensor.current. Si l'appareil renvoie 0 (cas fréquent sur Xiaomi/Samsung), le système bascule automatiquement sur l'estimation d'éclairage ARCore (requestedLightEstimation = AmbientIntensity) qui calcule la luminosité via les pixels de la caméra. Le seuil de bascule est fixé à **85 Lux** :
• Obscurité (<= 85 Lux) : Flash plein écran puissant (Alpha 90%), SpotLight 3D à intensité 6.5f, Torche LED du téléphone allumée.
• Pièce éclairée (> 85 Lux) : Flash adouci (Alpha 22%), SpotLight 3D à intensité 1.6f pour éviter l'éblouissement inutile.
2. **Gel des Zombies (Stun de 5 secondes) :** L'activation du bouton fige immédiatement tous les zombies (SetStunned(true)). La vitesse de leur composant Animator est mise à zéro (animator.speed = 0f) et leur script de mouvement s'arrête.
3. **Cooldown / Pause de 10 secondes :** Dès la fin des 5s de stun, le bouton entre en phase de recharge pendant 10 secondes. Le bouton se grise et affiche le décompte en direct : PAUSE 10s... 9s.... Tout spam est impossible jusqu'à la fin de la recharge.

2.5 NESSIM — CAPTEUR MICROPHONE, DÉTECTION DU SOL AR & COURSE DES ZOMBIES

Rôle : Capture audio microphone en temps réel, calcul du niveau sonore RMS, déclenchement de la course des zombies au moindre cri, calibrage rigoureux du sol horizontal AR.
Emplacement des fichiers :
• Gestionnaire Microphone : Assets/_Project/Scripts/Combat/ARMicDetectionManager.cs
• Calibrage du Sol AR : Assets/_Project/Scripts/Combat/ARZombieSpawner.cs
• Intelligence & Mouvement Zombie : Assets/Scenes/HamaZombieAI.cs

Fonctionnement technique :
1. **Détection du Sol Réel (Rejet des murs et des tables) :** Pour empêcher les zombies de flotter en l'air, ARZombieSpawner utilise TryFindLowestHorizontalFloorY(). Il filtre uniquement les surfaces horizontales orientées vers le haut (PlaneAlignment.HorizontalUp), exige que la surface soit au moins 70 cm sous le téléphone (ce qui rejette les tables et les chaises) et sélectionne la surface la plus basse de la pièce. Les zombies apparaissent entre 5.2m et 7.8m, les pieds parfaitement posés au sol.
2. **Détection Sonore par Microphone :** ARMicDetectionManager enregistre en boucle un flux microphone de 1 seconde. Il calcule l'amplitude efficace (RMS) en décibels. Le bruit ambiant normal est filtré. Si le joueur parle fort ou crie (au-delà du seuil thresholdDb = -24 dB), l'événement d'alerte se déclenche.
3. **Comportement des Zombies (Marche vs Course) :**
• État normal (Silence) : Le zombie marche calmement vers le joueur à vitesse modérée (**0.85 m/s**).
• État d'alerte (Bruit au micro) : Le zombie passe en sprint furieux à **4.2 m/s** avec rugissement, obligeant le joueur à réagir d'urgence !

3. FONCTIONNEMENT DES ANIMATIONS DU ZOMBIE (Ce que le professeur va demander)

Le modèle 3D du zombie utilise un composant **Animator** Unity standard associé à un **Animator Controller**. Voici comment les états d'animation sont pilotés par le script HamaZombieAI.cs :

État d'Animation	Condition de Déclenchement	Comportement & Code Associé
Idle (Attente)	À l'apparition avant activation	Le zombie est immobile au sol, orienté vers la caméra du joueur.
Walk (Marche normale)	État par défaut (Pas de bruit micro)	Vitesse : 0.85 à 0.95 m/s. Le zombie avance pas à pas en traquant la position du joueur en réalité augmentée.
Run (Course rapide)	Détecté par le micro (Nessim) quand le joueur crie	Vitesse : 4.2 m/s. Transition immédiate de l'Animator vers l'animation de sprint furieux (Z_Run_InPlace).
Attack (Attaque griffe)	Distance joueur <= 1.45 mètre	Le zombie s'arrête et assène des coups de griffes infligeant 15 dégâts toutes les 1.25 secondes.
Stunned (Gel complet)	Déclenché par le Flash de Mouhamed	Le script fait animator.speed = 0f : l'animation est suspendue dans sa frame exacte pendant 5 secondes.
Death (Mort)	Points de vie tombent à 0 (3 balles d'Amin)	Désactivation des colliders, animation de chute au sol, notification à QuestARLevelManager pour incrémenter le score.

4. AIDE-MÉMOIRE RAPIDE POUR LES QUESTIONS DU PROFESSEUR

• **Q : Comment avez-vous évité que les zombies flottent dans les airs ?**
R : Dans ARZombieSpawner.cs, on filtre les plans AR avec PlaneAlignment.HorizontalUp, on ignore les surfaces à moins de 70 cm sous le téléphone (tables) et on extrait le plan le plus bas (Lowest Y) clampé entre 1.15m et 1.80m.

• **Q : Pourquoi le capteur de lumière affichait 0 Lux et comment l'avez-vous résolu ?**
R : Sur Android, LightSensor.current renvoie souvent 0 car le driver matériel n'est pas routé par Unity. On a activé requestedLightEstimation = AmbientIntensity sur ARCameraManager pour lire la luminosité réelle en temps réel via la caméra ARCore.

• **Q : Comment fonctionne la transition entre le combat et l'énigme ?**
R : QuestARLevelManager compte les zombies tués. Dès que l'objectif est atteint, il vérifie que le joueur est vivant, lance une transition fluide et charge Mission5_Enigme avec le nombre de chiffres correspondant au niveau.

• **Q : Que se passe-t-il si le joueur meurt ?**
R : La transition est immédiatement annulée. ARGameOver.cs s'affiche en plein écran avec une vidéo immersive et les boutons Rejouer/Déconnexion.

5. GUIDE TECHNIQUE DU CODE SOURCE C# & QUESTIONS DE CODE PAR RÔLE

Cette section détaille les méthodes C# concrètes, les extraits de code source vérifiables et les questions pointues que le professeur peut poser à chaque membre sur l'implémentation algorithmique.

5.1 NEGZAOUI : CODE AUTHENTIFICATION, CAMÉRA & SESSION HUD

Extrait 1 : Résolution du conflit caméra sous Android (VisageCameraScanner.cs)

```
public void DisableBackgroundARCameras()
{
    // Empêche le crash CameraAccessException sur Android (deux caméras actives)
    GameObject arSessionGo = GameObject.Find("AR Session");
    if (arSessionGo != null && arSessionGo.activeSelf)
    {
        arSessionGo.SetActive(false);
        _disabledArObjects.Add(arSessionGo);
    }
    foreach (var b in FindObjectsByType<Behaviour>())
    {
        string t = b.GetType().Name;
        if (t == "ARSession" || t == "ARCameraManager" || t == "ARCameraBackground")
        {
            b.enabled = false;
            _disabledArBehaviours.Add(b);
        }
    }
}
```

Extrait 2 : Similarité cosinus 128-D multi-zones et seuil de 74% (MongoAtlasService.cs)

```
public static float ComputeCosineSimilarity(float[] a, float[] b)
{
    if (a == null || b == null || a.Length != b.Length) return 0f;

    // Découpage : Zone inférieure (barbe) + Zone supérieure (yeux/front) + Structure
    float lowerSim = ShiftedGridCosine(a, b, 0, 4, dr, dc);
    float upperSim = ShiftedGridCosine(a, b, 4, 10, dr, dc);
    float gridSim  = ShiftedGridCosine(a, b, 0, 10, dr, dc);

    // Pondération anatomique robuste aux ombres
    float combined = 0.45f * gridSim + 0.22f * upperSim + 0.16f * lowerSim + 0.17f * structSim;
    return combined; // Validé si score >= faceIdSimilarityThreshold (0.74 = 74%)
}
```

Extrait 3 : Session globale statique et affichage du joueur (VisageAuthModels.cs & ARPlayerSessionHud.cs)

```
// VisageAuthSession (Statique)
public static class VisageAuthSession
{
    public static MongoUserDocument CurrentUser { get; private set; }
    public static event Action<MongoUserDocument, string> OnUserLoggedIn;

    public static void SetSession(MongoUserDocument user, string method, float score = 1f) {
        CurrentUser = user;
        OnUserLoggedIn?.Invoke(user, method);
    }
}

// ARPlayerSessionHud.cs (Mise à jour du bandeau)
private void UpdatePlayerDisplay() {
    string name = VisageAuthSession.CurrentUser != null ? VisageAuthSession.CurrentUser.username :
"INVITÉ";
    playerLabel.text = $"JOUEUR : <color=#FF382E>{name}</color>";
}
```

Questions de code du Professeur pour Negzaoui :

• Q : Pourquoi avoir désactivé ARSession manuellement pendant le scan facial ?

R : Sur Android, l'accès au capteur photo physique est exclusif (Camera2 API). Ouvrir la caméra frontale selfie via WebCamTexture alors que la caméra arrière ARCore tourne en tâche de fond produit une exception critique et un écran noir. Notre méthode coupe temporairement l'ARSession et la rétablit proprement après le login.

• Q : Comment le nom du joueur persiste-t-il entre la scène de login et le jeu AR ?

R : Grâce à la classe statique C# VisageAuthSession. Les variables statiques restent allouées en mémoire vive tout au long de la durée de vie du processus Unity, même lors d'un SceneManager.LoadScene().

5.2 AMIN : CODE ARMES, BALISTIQUE & CALCUL DES DÉGÂTS

Extrait 1 : Détection balistique et fenêtre anti-double-hit (HamaZombieAI.cs)

```
private void TryProcessProjectileHit(GameObject hitObj)
{
    if (isDead || hitObj == null || QuestARLevelManager.IsPlayerDead()) return;

    if (hitObj.GetComponent("Projectile") != null ||
        hitObj.name.IndexOf("bullet", System.StringComparison.OrdinalIgnoreCase) >= 0)
    {
        // Fenêtre de garde anti-double impact (60 ms)
        if (Time.time - lastBulletHitTime < 0.06f) {
            Destroy(hitObj);
            return;
        }

        lastBulletHitTime = Time.time;
        Destroy(hitObj);     // Destruction du projectile
        TakeDamage(35f);     // 35 dégâts par balle (Zombie = 100 HP => 3 balles pour le tuer)
    }
}
```

Extrait 2 : Coroutine de swap d'arme animée (ARInfimaWeaponSwitcher.cs)

```
IEnumerator SwapRoutine()
{
    swapping = true;
    characterAnimator?.BeginHolster(); // Animation ranger l'arme
    yield return WaitForStage(1.25f);

    int next = inventory.GetNextIndex();
    inventory.Equip(next);             // Change d'index dans l'inventaire
    characterAnimator?.SetWeapon(inventory.GetEquipped());
    characterAnimator?.PlayUnholster(); // Animation sortir la nouvelle arme
    yield return WaitForStage(1.4f);

    swapping = false;
}
```

Questions de code du Professeur pour Amin :

• Q : Pourquoi avez-vous mis un délai de 0.06s sur la collision des balles ?

R : Un modèle de zombie possède plusieurs colliders (buste, tête, capsule globale). Si une balle pénètre le collider physique et le trigger d'alerte dans la même frame, le script appliquerait 70 dégâts au lieu de 35. Ce timer lastBulletHitTime garantit 1 balle = 1 seul impact.

5.3 SOUHAILA : CODE ÉNIGME, GAZE TRACKING & FILTRAGE DYNAMIQUE

Extrait 1 : Visée par le regard sans toucher l'écran (IndicesFlottants.cs)

```
Vector3 dir = ind.racine.position - cam.transform.position;
float angle = Vector3.Angle(cam.transform.forward, dir);

// Tolérance de visée centrale < 8 degrés
if (angle < AngleMax) {
    ind.temps += Time.deltaTime;
} else {
    ind.temps = 0f;
}

// Remplissage de l'anneau holographique
progressionVisee = Mathf.Max(progressionVisee, Mathf.Clamp01(ind.temps / DureeRegard));

// Validation automatique au bout de 0.8 seconde de fixation
if (ind.temps >= DureeRegard || ind == touche) {
    Lire(System.Array.IndexOf(indices, ind));
}
```

Extrait 2 : Adaptation de la taille du code selon le niveau (IndicesFlottants.cs)

```
int reqDigits = Mathf.Clamp(QuestARLevelManager.GetPuzzleDigitsForCurrentLevel(), 2, 4);
int maxLeurres = reqDigits == 2 ? 1 : (reqDigits == 3 ? 2 : 3);

// Niveau 1 -> 2 chiffres (ex: 47) + 1 leurre (piège)
// Niveau 2 -> 3 chiffres (ex: 472) + 2 leurres
// Niveau 3 -> 4 chiffres (ex: 4729) + 3 leurres
foreach (RelicData r in enigme.reliques) {
    if (r.authentique && r.rang <= reqDigits) filteredList.Add(r);
    else if (!r.authentique && leurresAdded < maxLeurres) {
        filteredList.Add(r);
        leurresAdded++;
    }
}
```

Questions de code du Professeur pour Souhaila :

• Q : Comment orientez-vous les reliques 3D vers le joueur ?

R : Dans l'Update() de chaque relique, on calcule le vecteur dir = racine.position - cam.position puis on applique Quaternion.LookRotation(dir) pour que les chiffres et symboles restent toujours lisibles face caméra.

5.4 MOUHAMED : CODE FLASH STUN, COOLDOWN 10s & CAPTEUR LUMIÈRE

Extrait 1 : Gel des zombies et gestion du cooldown de 10s (HamaFlashStunManager.cs)

```
// Gel immédiat de tous les zombies
HamaZombieAI[] zombies = Object.FindObjectsByType<HamaZombieAI>(FindObjectsInactive.Exclude);
foreach (HamaZombieAI z in zombies) {
    if (z != null) z.SetStunned(true); // Coupe animator.speed = 0f
}
SetARFoundationTorch(true); // Allume la torche LED physique du smartphone

float remaining = stunDuration; // 5 secondes
while (remaining > 0f) {
    remaining -= Time.deltaTime;
    yield return null;
}

SetARFoundationTorch(false);
foreach (HamaZombieAI z in zombies) {
    if (z != null) z.SetStunned(false); // Rétablit animator.speed = 1f
}

// Début du temps de pause / cooldown de 10 secondes
remainingCooldownTime = cooldownDuration; // 10.0f
```

Extrait 2 : Système hybride capteur physique et fallback ARCore (HamaFlashStunManager.cs)

```
// 1. Capteur matériel du smartphone (LightSensor)
if (LightSensor.current != null) {
    float sensorLux = LightSensor.current.lightLevel.ReadValue();
    if (sensorLux > 0.05f) {
        hasValidPhysicalReading = true;
        currentAmbientLux = Mathf.Lerp(currentAmbientLux, sensorLux, Time.deltaTime * 6f);
    }
}

// 2. Fallback automatique ARCore
private void OnCameraFrameReceived(ARCameraFrameEventArgs args) {
    if (args.lightEstimation.averageBrightness.HasValue) {
        float b = args.lightEstimation.averageBrightness.Value; // 0.0f à 1.0f
        float estimatedLux = Mathf.Clamp(b * 260f, 15f, 400f);
        if (!hasValidPhysicalReading) {
            currentAmbientLux = Mathf.Lerp(currentAmbientLux, estimatedLux, 0.35f);
        }
    }
}
```

Questions de code du Professeur pour Mouhamed :

• Q : Comment empêchez-vous les zombies d'attaquer pendant le flash ?

R : SetStunned(true) met animator.speed = 0f et force la variable d'état isStunned = true. Dans la boucle de déplacement et d'attaque du zombie, un garde if (isStunned) return; neutralise tout mouvement et calcul de dégâts.

5.5 NESSIM : CODE DÉTECTION MICROPHONE (RMS/dB) & CALIBRAGE SOL AR

Extrait 1 : Formule RMS et conversion logarithmique en décibels (ARMicDetectionManager.cs)

```
// Lecture du buffer microphone (1024 échantillons)
micClip.GetData(sampleBuffer, startPosition);

// 1. Puissance quadratique moyenne efficace (Root Mean Square)
float sum = 0f;
for (int i = 0; i < samplesToRead; i++) {
    float s = sampleBuffer[i];
    sum += s * s;
}
currentRms = Mathf.Sqrt(sum / samplesToRead);

// 2. Soustraction du bruit ambiant et conversion en décibels (dB)
float netRms = Mathf.Max(0f, currentRms - noiseFloorRms * 1.35f);
currentDecibels = 20f * Mathf.Log10(Mathf.Max(netRms * amplificationGain, 0.00001f));

// Déclenchement de l'événement si le seuil de -24 dB est dépassé
if (currentDecibels >= thresholdDb && isClearlyAboveAmbient) {
    sustainedLoudTimer += Time.deltaTime;
    if (sustainedLoudTimer >= 0.14f) {
        OnLoudSound?.Invoke(); // Sprint furieux des zombies !
    }
}
```

Extrait 2 : Protection anti-larsen lors des tirs d'armes (ARMicDetectionManager.cs)

```
public static void NotifyWeaponShot()
{
    // Ignore l'écoute pendant 0.42 seconde lors d'un coup de feu
    // Empêche le haut-parleur du téléphone de déclencher le cri du micro
    ignoreUntilTime = Time.time + 0.42f;
}
```

Extrait 3 : Algorithme de calibrage sol et rejet des tables (ARZombieSpawner.cs)

```
private bool TryFindLowestHorizontalFloorY(out float floorY)
{
    float camY = playerCam.position.y;
    floorY = camY - DefaultCameraToFloorHeight;

    // Filtre 1 : Plans horizontaux orientés vers le haut (HorizontalUp)
    // Filtre 2 : Rejet des tables et meubles hauts (hitY <= camY - 0.70m)
    if (hitY <= camY - 0.70f) {
        floorY = hitY;
    }

    // Filtre 3 : Sélection du plan le plus bas clampé entre 1.15m et 1.80m sous le téléphone
    floorY = Mathf.Clamp(floorY, camY - 1.80f, camY - 1.15f);
    return foundFloor;
}
```

Extrait 4 : Transition immédiate marche vers course à 4.2 m/s (HamaZombieAI.cs)

```
private void HandleLoudSound()
{
    alertTimer = alertDuration; // 5.0 secondes d'alerte
    currentState = ZombieState.Running;
}

// Dans l'Update :
float activeSpeed = (currentState == ZombieState.Running) ? 4.2f : 0.95f; // 4.2 m/s vs 0.95 m/s
transform.position += moveDir * (activeSpeed * Time.deltaTime);

if (currentState == ZombieState.Running) {
    PlayState("Z_Run_InPlace"); // Animation course rapide
    roarAudio.Play();           // Rugissement agressif
}
```

Questions de code du Professeur pour Nessim :

• Q : Pourquoi soustraire le bruit ambiant (noiseFloorRms) avant de convertir en dB ?

R : Une pièce de soutenance peut être bruyante (projecteurs, ventilation, discussions). Calibrer le bruit de fond pendant la première seconde et le soustraire permet d'obtenir un niveau relatif où le silence de la pièce donne toujours environ -80 dB, rendant le seuil de cri de -24 dB fiable quel que soit le lieu.

6. SYNTHÈSE DE CONFORMITÉ & CONCLUSION TECHNIQUE

Le projet **QuestAR** a fait l'objet d'une campagne complète de consolidation technique :
• **Code C# :** Strictement conforme aux standards Unity 6 & URP, compilant à 100% avec 0 erreur.
• **Hardware Mobile :** Utilisation équilibrée des capteurs réels (Caméra, Micro, Capteur de luminosité) avec mécanismes de repli logiciel garantissant la robustesse.
• **Architecture :** Découpage modulaire par responsabilités individuelles, facilitant la soutenance et la vérification du travail de chaque étudiant.